

## Highlights

**Does energy-efficient WSN routing need learning? A benchmark against the provable optimum and a coordinated Dijkstra over a shared load table**

[Author 1], [Author 2], [Supervisor]

- Learned and heuristic WSN routers are scored against a provable life-time optimum.
- A recent deep RL router reaches 11% of the optimum; battery Dijkstra reaches 88%.
- LEST load-table Dijkstra books load within each round and needs no solver.
- It wins 118 of 120 time-varying deployments and holds on real Intel Lab traffic.
- Forecasts add 0.5 points to an LP planner and nothing to Dijkstra.

# Does energy-efficient WSN routing need learning? A benchmark against the provable optimum and a coordinated Dijkstra over a shared load table

[Author 1]<sup>a,\*</sup>, [Author 2]<sup>a</sup>, [Supervisor]<sup>a</sup>

<sup>a</sup>[Department], Netaji Subhas University of Technology, New Delhi, 110078, India

---

## Abstract

Deep reinforcement learning is increasingly proposed for energy-efficient routing in wireless sensor networks (WSNs), but learned routers are rarely compared with strong classical baselines or with the achievable optimum. We score every method against a provable lifetime ceiling, the Chang–Tassiulas maximum-lifetime linear program, and an oracle version for time-varying traffic. On held-out deployments in the setting of a recent multi-agent deep reinforcement learning router (MADII, IEEE Internet of Things Journal, 2025), MADII reaches 11% of the ceiling. Battery-weighted Dijkstra reaches 88% with no training, and reinforcement learning warm-started from it adds nothing significant. We then propose LEST load-table Dijkstra, a solver-free coordinated router. Each round the sink builds the routing tree node by node, nearest the sink first, and charges every relay for the traffic already booked through it in that round. Exact batteries are combined with node loads shared as 4-tier entries piggybacked on existing packets, and this overhead is charged in the energy model. Under four time-varying traffic scenarios it reaches 90.5% of the oracle, against 85.0% for battery-weighted Dijkstra (118 of 120 deployments won; Wilcoxon  $p < 10^{-20}$ ). A published energy-and-load router (EBR-DA) reaches 15%. An LP planner re-solved every round reaches 97.0% and serves as an upper reference; a traffic forecast adds only 0.5 points to it. The ranking holds on event-driven traffic derived from the Intel Berkeley Lab dataset (MADII 11.8%, battery Dijkstra 86.0%, LEST Dijkstra 90.8%, LP planner 96.9%).

---

\*Corresponding author.

Email address: [email] ([Author 1])

*Keywords:* Wireless sensor networks, Network lifetime, Energy-efficient routing, Load balancing, Maximum-lifetime linear program, Deep reinforcement learning, Benchmarking

---

## 1. Introduction

A wireless sensor network (WSN) consists of battery-powered nodes that sense their surroundings and forward readings to a sink, usually over several hops. Every packet a node sends or relays costs energy, and once the first node is exhausted part of the network can be cut off. The number of rounds until the first node dies (FND) is therefore a standard measure of network lifetime [1], and lifetime-aware routing is a central design problem in WSNs and in the low-power Internet of Things (IoT).

A growing line of work applies deep reinforcement learning (DRL) to this problem: neural networks learn to choose next hops, cluster heads or transmit powers [2, 3, 4]. These methods need long training and often a GPU. They are usually evaluated against clustering heuristics, metaheuristics or other learned methods. We found no recent learned WSN router that was compared with a battery-weighted shortest-path router, and none that reported its lifetime as a fraction of a provable optimum. Recent comparison studies even tabulate published results obtained on different datasets and in different units [5]. As a result, it is hard to tell how much the learning contributes. Neighbouring areas of networking show the same pattern: simple, well-tuned baselines match deep models once they are evaluated carefully [6].

This paper answers the question by measurement. The maximum-lifetime linear program (LP) of Chang and Tassiulas [1] gives a ceiling that no routing scheme can exceed. For traffic that varies over time we use an oracle version that knows all future traffic. Every method is scored as the percentage of that ceiling it reaches, on held-out deployments and on real traffic. Against this yardstick the methods separate into clear levels. A recent multi-agent DRL router, MADII [2], rebuilt in its own published setting, reaches about 11–15% of the ceiling. A published energy-and-load router, EBR-DA [7], reaches about 15%. Battery-weighted Dijkstra reaches 85–88% with no training.

The remaining gap comes from coordination: in battery-weighted Dijkstra every node picks its path independently, so in the same round many nodes can pick the same relay. We propose *LEST load-table Dijkstra*, a solver-free router that closes about half of the gap. Each round the sink builds the

routing tree one node at a time, nearest the sink first. Each node’s path is found by Dijkstra over a link cost that combines radio energy, the exact residual battery and the load already booked on each relay in that round. Each source’s traffic rate reaches the sink through a Lightweight State Table (LEST): every node reports its load as one of four tiers, piggybacked on packets it already sends. Finally, an LP planner re-solved every round from live batteries serves as an upper reference for what full flow optimisation adds.

*Contributions.* The contributions of this paper are:

- (C1) **An oracle-normalised benchmark for lifetime routing.** Every method is reported as a percentage of a provable ceiling: the maximum-lifetime LP for static traffic, and an oracle LP, solved by bisection, for time-varying traffic. The benchmark covers 30 held-out deployments per scenario, four traffic scenarios, real event-driven traffic from the Intel Berkeley Lab dataset [8] and paired Wilcoxon tests with Holm correction.
- (C2) **A measured ranking of learned, published and classical routers.** MADII, rebuilt in its own setting and checked against its published figures, reaches 11.1% of the ceiling (19.5 rounds against the paper’s 19). Battery-weighted Dijkstra reaches 88.4%, and DRL warm-started from Dijkstra adds no significant gain ( $p = 0.24$ ).
- (C3) **LEST load-table Dijkstra.** A per-round, solver-free and training-free router that builds the tree sequentially with within-round load booking, using exact energy and a 4-tier shared load table whose overhead is charged in the energy model. It reaches 90.5% of the oracle against 85.0% for battery-weighted Dijkstra (118/1/1 wins, ties and losses over 120 deployments) and 90.8% against 86.0% on Intel Lab traffic.
- (C4) **Ablations and negative results.** Both the energy term and the load term are needed (load only 45.1%, energy only 85.0%, both 90.5%), and energy must stay exact rather than tiered. Five ways of adding prediction, lifetime weighting or LP prices to Dijkstra stay within about  $\pm 1$  point of it. A Holt-Winters forecast adds only 0.5 points to an LP planner that is re-solved every round.

All code, the result files and the scripts that generate every table and figure are released (see the data availability statement).

Section 2 reviews related work and positions this paper. Section 3 states the system model, the packet-layer model and every assumption. Section 4 describes the proposed router and the reference methods, and Section 5 the evaluation protocol. Section 6 reports the results and the statistical tests, Section 7 discusses them together with the threats to validity, and Section 8 concludes.

## 2. Related work and literature survey

### 2.1. Energy models and clustering

Heinzelman et al. [9] introduced the first-order radio model that most WSN routing studies use, including ours (Section 3.2). In it, transmitting costs electronics energy plus an amplifier term that grows with the square or the fourth power of distance, and receiving costs electronics energy only. Their LEACH protocol popularised clustering with rotating cluster heads. Later schemes select cluster heads with fuzzy c-means (FCM) or with population metaheuristics such as grey-wolf/whale hybrids, the honey badger algorithm (HBA) and the pelican optimisation algorithm (POA). These are the baselines against which MADII is evaluated [2].

### 2.2. Lifetime-optimal and energy-aware shortest-path routing

Dijkstra’s algorithm [10] finds least-cost paths for given link costs. With radio energy as the cost it gives minimum-total-energy (MTE) routing, which is optimal for one round but exhausts the same relays first. Chang and Tassiulas [1] formulated maximum-lifetime routing as an LP over link flows. Because the LP allows fractional, time-shared routing, its optimum is an upper bound on the lifetime of any routing scheme, and we use it as the ceiling. They also proposed flow augmentation: shortest paths over link costs that combine energy per bit with the residual energy at both ends, recomputed as flow is added. They reported lifetimes close to the LP optimum and compared against MTE routing. The battery-weighted Dijkstra baseline in this paper is the single-tree, per-round member of that family. Recent theoretical progress on single-source shortest paths [11] computes the same paths asymptotically faster. It cannot change routing quality, and at WSN sizes the cost of Dijkstra itself is negligible (Section 6.7).

### 2.3. Load-balanced routing trees

Traffic funnels towards the sink, so the nodes next to it relay the most and die first. Dai and Han [12] proposed a node-centric algorithm that grows a tree from the sink, repeatedly grafting the heaviest unassigned border node onto the lightest branch. As described in [13], it balances load but ignores distance and energy. Shan et al. [13] construct a load-balanced *shortest-path* spanning tree, in which every sensor reaches the sink in its minimum hop count. Their top-down network-flow construction is followed by distributed balance refinement, and they report lifetimes of about 85% of an upper bound. Both build one tree under uniform initial energy; neither rebuilds it from residual batteries. EBR-DA [7] combines residual energy and buffer occupancy in a node weight over a hop tree, and each node forwards to its lightest neighbour one hop nearer the sink. It is evaluated in MATLAB against DRINA and InFRA, two aggregation-oriented protocols, with no optimum bound. Combining energy and load in a routing cost, and building load-balanced trees, are therefore not new. What we add is the per-round rebuild with exact battery weighting, the within-round booking order, the LEST sharing with charged overhead, and the measurement against the optimum. Our ablations (Section 6.3) show that the load term alone, close in spirit to load-only trees, reaches only 45% of the oracle.

### 2.4. Learning-based routing and energy control

Tang et al. [4] propose a two-tier system for low-power mesh networks: a central reinforcement-learning coordinator plus small neural networks on each node that adjust transmit power. MADII [2] is a multi-agent deep Q-network [14] with an Informer encoder [15], which picks the next hop of every sensor under centralised training and centralised execution. It is trained first from clustering algorithms through imitation and then by exploration, and reports gains over FCM, GWO-WOA, HBA, POA and learned ablations. The full text contains no shortest-path baseline and no optimum bound. Younus et al. [3] train an RL agent inside a software-defined WSN controller that has the global network view. Their testbed evaluation compares the agent with an RL-based WSN routing scheme and reports a 23–30% longer lifetime. A recent review of learning for routing [16] surveys the broader trend. In each case the learned method is compared with heuristics or other learners, not with battery-weighted shortest paths or the optimum.

### 2.5. Forecasting and algorithms with predictions

Holt-Winters exponential smoothing [17, 18] forecasts series with level, trend and seasonal components, and it remains a strong baseline for periodic data such as daily sensor traffic. Work on algorithms with predictions [19] studies classical algorithms that take a forecast as advice. It asks how much they gain when the forecast is right and how much they lose when it is wrong. We follow this framing when we test forecasts inside Dijkstra and inside an LP planner (Section 6.4).

### 2.6. Literature survey and research gap

Table 1 summarises the closest studies. Table 2 compares their characteristics with this work: what each router uses to decide, and how each study is evaluated. A recent comparison paper [5] illustrates the evaluation gap. It tabulates the published results of about ten methods on two different datasets, in units ranging from joules to percentages, with no common simulator, no shortest-path baseline and no optimum. The gap that this paper fills is first one of measurement: learned and heuristic WSN routers are published without reference to the simplest strong classical router or to the achievable lifetime. It is second one of method: no solver-free router that we know of combines exact battery weighting with load booked within the round and a shared tiered load table, re-plans every round, and charges its own overhead.

## 3. System model and assumptions

### 3.1. Network model

The network has  $n$  static sensor nodes  $V = \{1, \dots, n\}$  and one sink  $s$ . In the main setting, which is MADII’s published setting [2],  $n = 100$  nodes are placed uniformly at random in a  $500 \times 500$  m field with the sink at the centre. Every node starts with energy  $E_0$ . Time is divided into rounds. In round  $t$  node  $i$  generates  $k_i(t)$  packets of  $L$  bits, and every packet is forwarded to the sink along the round’s routing tree, described by a parent function  $p_t : V \rightarrow V \cup \{s\}$ . Relays forward everything they receive, with no aggregation. The network lifetime  $T$  is the number of rounds until the first node exhausts its battery (FND). We also report the rounds until 10%, 25% and 50% of the nodes are dead, the data delivered, energy efficiency (kbit delivered per joule) and hops per packet.

Table 1: Literature survey of the closest work.

| Work (year)                     | Category                        | Approach                                                               | Compared with                            | Gap relative to this work                                      |
|---------------------------------|---------------------------------|------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------------------|
| Heinzelman et al. [9] (2000)    | Clustering                      | LEACH; first-order radio model                                         | Direct, MTE                              | No optimum; clustering rather than multi-hop routing to a sink |
| Dai and Han [12] (2003)         | Load-balanced tree              | Node-centric grafting of heavy nodes onto light branches               | – (as described in [13])                 | Load only; no energy or distance; one tree                     |
| Chang and Tassiulas [1] (2004)  | Lifetime-optimal routing        | Max-lifetime LP; flow augmentation over energy-and-residual link costs | LP optimum, MTE                          | No real data, overhead or statistical tests; table-driven      |
| Shan et al. [13] (2013)         | Load-balanced tree              | Load-balanced shortest-path tree by network flow plus refinement       | Node-centric [12], upper bound           | One static tree; uniform energy; min-hop constraint            |
| Mahdi et al., EBR-DA [7] (2018) | Energy + load routing           | Hop tree; node weight $0.6(1 - E/E_0)^2 + 0.4 \text{ buffer}^2$        | DRINA, InFRA                             | No optimum or shortest-path baseline; designed for aggregation |
| Younus et al. [3] (2021)        | Learned (SDWSN)                 | RL in a controller with a global view; four rewards                    | RL-based WSN routing                     | No classical baseline or optimum; testbed only                 |
| Tang et al. [4] (2025)          | Learned power control           | Central RL coordinator plus per-node networks                          | –                                        | Power control rather than routing; no optimum                  |
| Yang et al., MADII [2] (2025)   | Learned routing                 | Multi-agent DQN with Informer; imitation then exploration              | FCM, GWO-WOA, HBA, POA, ablations        | No shortest-path baseline or optimum; constant traffic         |
| ICMCSI study [5] (2026)         | Comparison                      | Tabulates ten published methods on two datasets                        | Published numbers                        | Mixed units; no common simulator or bound                      |
| Duan et al. [11] (2025)         | Shortest-path theory            | SSSP faster than sorting-bound Dijkstra                                | Theory                                   | Same paths, so no change to lifetime                           |
| This work                       | Benchmark + coordinated routing | Oracle-normalised bench; LEST load-table Dijkstra; LP planner          | Optimum, battery Dijkstra, MADII, EBR-DA | –                                                              |

Table 2: Characteristics of the closest work and of this work. ✓: provided; ∼: partly; ✗: not provided or not reported. Method columns describe what the router uses to decide; evaluation columns describe the study.

| Work                                            | Method |    |    |    |    |    | Evaluation |    |    |    |    |    |
|-------------------------------------------------|--------|----|----|----|----|----|------------|----|----|----|----|----|
|                                                 | M1     | M2 | M3 | M4 | M5 | M6 | E1         | E2 | E3 | E4 | E5 | E6 |
| Chang and Tassiulas [1]                         | ✓      | ∼  | ∼  | ✓  | ✓  | ✓  | ✓          | ✗  | ✓  | ✓  | ✗  | ✗  |
| Shan et al. [13]                                | ✗      | ✓  | ✓  | ✗  | ✓  | ∼  | ✗          | ∼  | ∼  | ✓  | ✗  | ✗  |
| EBR-DA [7]                                      | ✓      | ✓  | ✗  | ✓  | ✓  | ✓  | ∼          | ✗  | ✗  | ✗  | ✗  | ✗  |
| Younus et al. [3]                               | ✓      | ✓  | ∼  | ✓  | ✗  | ✓  | ✗          | ∼  | ✗  | ✗  | ✓  | ✗  |
| MADII [2]                                       | ✓      | ✓  | ∼  | ✓  | ✗  | ✓  | ✗          | ∼  | ✗  | ✗  | ✗  | ✗  |
| Battery-weighted Dijkstra (baseline, this work) | ✓      | ✗  | ✗  | ✓  | ✓  | ✓  | ✓          | ✓  | ✓  | ✓  | ✓  | ✓  |
| LP planner (upper reference, this work)         | ✓      | ✓  | ✓  | ✓  | ✓  | ✗  | ✓          | ✓  | ✓  | ✓  | ✓  | ✓  |
| <b>LEST load-table Dijkstra (proposed)</b>      | ✓      | ✓  | ✓  | ✓  | ✓  | ✓  | ✓          | ✓  | ✓  | ✓  | ✓  | ✓  |

M1 residual energy in the route decision; M2 relay load in the route decision; M3 within-round coordination (a node’s choice accounts for the choices of others in the same round); M4 re-planned from the current state every round; M5 no offline training; M6 no optimisation solver at run time.

E1 time-varying traffic evaluated; E2 control overhead charged in the energy model; E3 compared with a shortest-path baseline; E4 compared with a provable optimum; E5 real data or testbed (for this work, real Intel Lab traffic traces); E6 statistical significance tests.

Table 3: Network, radio and packet parameters (MADII’s Table I [2]).

| Parameter                                            | Value                                                   |
|------------------------------------------------------|---------------------------------------------------------|
| Sensors, field, sink                                 | 100, uniform in $500 \times 500$ m, centre              |
| Initial energy $E_0$                                 | 0.5 J (Intel Lab bench: 1.0 and 1.5 J)                  |
| Data packet $L$                                      | 4,000 bits                                              |
| Route advertisement $L_c$                            | 100 bits per round, to every node                       |
| $E_{\text{elec}}$                                    | 50 nJ/bit                                               |
| $\varepsilon_{\text{fs}} / \varepsilon_{\text{amp}}$ | 10 pJ/bit/m <sup>2</sup> / 0.0013 pJ/bit/m <sup>4</sup> |
| Crossover distance $d_0$                             | 87.7 m                                                  |
| Aggregation at relays                                | None (full forwarding)                                  |
| LEST load report                                     | +8 bits on each data packet                             |
| LEST snapshot                                        | +2 bits per node in the advertisement                   |

### 3.2. Radio energy model

Transmitting  $k$  bits over distance  $d$  and receiving  $k$  bits cost

$$E_{\text{TX}}(k, d) = \begin{cases} kE_{\text{elec}} + k\varepsilon_{\text{fs}}d^2, & d < d_0, \\ kE_{\text{elec}} + k\varepsilon_{\text{amp}}d^4, & d \geq d_0, \end{cases} \quad (1)$$

$$E_{\text{RX}}(k) = kE_{\text{elec}}, \quad d_0 = \sqrt{\varepsilon_{\text{fs}}/\varepsilon_{\text{amp}}} \approx 87.7 \text{ m}. \quad (2)$$

We write  $e_{\text{tx}}(i, j) = E_{\text{TX}}(1, d_{ij})$  and  $e_{\text{rx}} = E_{\text{elec}}$  for the per-bit costs. Table 3 lists the parameters.

### 3.3. Packet-layer model

The study is round-based rather than slot-based, but every bit a router adds is charged at the packet layer, so the routers pay for their own coordination. Three packet types exist:

- (i) **Data packets** of  $L = 4,000$  bits. Each packet is sent hop by hop to the sink. At every hop the sender pays  $E_{\text{TX}}(L, d)$  and the receiver  $E_{\text{RX}}(L)$ .
- (ii) **The route advertisement**, a broadcast of  $L_c = 100$  bits that the sink sends once per round. Every alive node pays  $E_{\text{RX}}(L_c)$  to receive its next hop. This is the control cost MADII’s setting already charges, and it is identical for every method.

- (iii) **LEST traffic**, charged only to the LEST router. Each data packet carries an extra byte holding the sender’s load report, so  $L$  becomes 4,008 bits for every hop of every packet. The advertisement carries the snapshot of all load tiers, 2 bits per node, so  $L_c$  becomes  $100 + 2n = 300$  bits at  $n = 100$ .

A relay that cannot afford a transmission dies within the round, and the packets routed through it in that round are lost. A source whose path fails retries directly to the sink if its battery allows. The delivery ratio is reported with the results (Table 11). No acknowledgements, retransmissions or MAC frames are modelled, and the MAC is assumed to schedule transmissions without collisions; see assumption A5 below. This is MADII’s idealisation, and we keep it so that the comparison is made on MADII’s own terms.

### 3.4. Traffic models

*Static traffic.* Every node sends one packet per round,  $k_i(t) = 1$ , as in MADII.

*Time-varying traffic.* One day is 24 rounds, and every method gets 3 days of pre-deployment history. There are four scenarios:

- **Daily surge (R).** The 20% of nodes nearest a random point form a hotspot. They send Poisson traffic with mean 4 packets per round during a 6-round evening and 0.5 otherwise; all other nodes have mean 1.
- **Constant (N).**  $k_i(t) = 1$  for every node and round.
- **Bursts (B).** R plus random bursts: each round, with probability 0.05, a random node sends five times its rate for 4 rounds.
- **Pattern shift (D).** R with the hotspot moved to 20 different nodes after two days. D is reported as a robustness test, and nothing is tuned on it.

*Real traffic (Intel Berkeley Lab).* The Intel Lab dataset [8] holds about 2.3 million readings from 54 motes. Traffic is derived by send-on-delta reporting with one-hour rounds. Each mote sends one heartbeat per hour, plus one packet whenever its temperature or light has moved by more than 0.5 °C or 100 lux since its last report. This gives about 3.4 packets per mote-hour, and the busiest hour carries 6.6 times the traffic of the quietest. Readings from motes below 2.4 V or with temperatures outside 0–50 °C are dropped. The 31 days before most motes died are used. The real mote layout (40 × 31 m) is scaled by 12 to MADII’s field, because at lab scale the direct link to the sink is almost always cheapest.

### 3.5. Assumptions

All results hold under the following assumptions. A1–A8 are those of MADII’s setting [2], which we keep so that the comparison is fair. A9–A15 make explicit what that setting leaves implicit.

- A1 Static, homogeneous nodes.** Nodes do not move. All start with the same energy  $E_0$ ; there is no energy harvesting and no battery replacement.
- A2 One static sink with unlimited resources.** The sink is mains-powered. Its computation (Dijkstra, LP solves) is not charged to any sensor.
- A3 Free-space terrain, no barriers.** There are no obstacles, walls or terrain effects. Link cost depends only on Euclidean distance through Eqs. (1)–(2).
- A4 Full reachability with power control.** Any node can reach any other node and the sink by adjusting its transmit power, paying the distance-dependent cost. The exception is EBR-DA, which is restricted to the 80 m range its paper specifies.
- A5 Ideal channel and MAC.** There is no packet loss from fading, noise, interference or collisions, and no retransmissions or acknowledgements. Links are symmetric.
- A6 Radio-only energy.** Only transmission and reception of bits consume energy. Sensing, processing, idle listening and sleep are not modelled.
- A7 No aggregation or compression.** Relays forward every bit they receive.
- A8 Round-based operation.** Routes are fixed within a round. The sink computes the round’s routing before the round and broadcasts it in the 100-bit advertisement, which every alive node receives. Rounds are synchronised.
- A9 Known positions.** The sink knows every node’s position, and hence every link cost.

- A10 Known residual energy.** Every method that uses energy sees each node’s exact residual battery at the start of each round. This is assumed free and identical for all methods, so it adds no bias between them.
- A11 Observed traffic only.** The sink learns each node’s packet count for round  $t$  when those packets arrive, after the round. No method sees future traffic except the oracle ceiling and the explicitly labelled perfect-forecast diagnostic.
- A12 Failure only by depletion.** Nodes fail only by running out of energy. There are no hardware faults, no mobility and no security attacks.
- A13 Failure mechanics.** A relay that cannot afford its transmission dies within the round, and the traffic through it is lost; affected sources retry directly to the sink if they can.
- A14 Shared randomness.** Deployments are drawn from fixed seeds, and every method sees the same deployments and traffic, so all comparisons are paired.
- A15 Scaled real layout.** On the Intel Lab bench the layout is scaled to the 500 m field and traffic is derived from sensor readings. Measured radio traffic and the dataset’s connectivity are not used.

The idealised channel (A3–A5) is the main limitation, and Section 7.1 discusses its consequences.

### 3.6. Problem statement and lifetime ceiling

A routing policy chooses  $p_t$  every round from the information available before the round (A8–A11). The goal is to maximise  $T$ . Let  $f_{ij} \geq 0$  be the total bits sent on link  $i \rightarrow j$  over the whole lifetime. For static traffic the maximum lifetime  $T^*$  solves the LP [1]

$$\begin{aligned} \max_{T, f \geq 0} \quad & T \quad \text{subject to, for every } i \in V, \\ \sum_j f_{ij} - \sum_k f_{ki} &= L T, \end{aligned} \tag{3}$$

$$\sum_j f_{ij} e_{\text{tx}}(i, j) + \sum_k f_{ki} e_{\text{rx}} + T E_{\text{RX}}(L_c) \leq E_0. \tag{4}$$

Because the LP allows fractional, time-shared routing, any sequence of trees that keeps every node alive for  $T$  rounds gives a feasible point, so  $T \leq T^*$ .

For time-varying traffic the ceiling is an oracle that knows all future traffic. It is the largest  $T$  for which a flow carries every node’s first- $T$ -rounds traffic within its battery:

$$\sum_j f_{ij} - \sum_k f_{ki} = L \sum_{t < T} k_i(t), \quad (5)$$

with the energy constraint (4) unchanged, found by bisection on  $T$ . Every method is scored as  $100 T/T^*$ , the percentage of its deployment’s ceiling it reaches. The ceiling was verified in two ways. On small networks, exhaustive enumeration of every acyclic routing tree never exceeds it. On every 100-node deployment, no policy exceeded it.

## 4. Methods

All routers compute the round’s tree at the sink from the information listed in A8–A11 and broadcast it in the advertisement. Nothing new runs on the sensors.

### 4.1. Battery-weighted Dijkstra

Each round, Dijkstra [10] finds the least-cost path from every node to the sink over the link cost

$$w(i, j) = [e_{\text{tx}}(i, j) + e_{\text{rx}}] \cdot \frac{E_0}{E_i}, \quad (6)$$

where  $E_i$  is the sender’s residual energy. The ratio  $E_0/E_i$  rises sharply as a node nears empty, so its children move elsewhere. Dropping the factor gives min-energy (MTE) Dijkstra. A sweep of the exponent  $(E_0/E_i)^\alpha$  on validation deployments peaks at  $\alpha = 1$  (Section 6.4).

### 4.2. Proposed: LEST load-table Dijkstra

In battery-weighted Dijkstra every node’s path is computed independently, so in one round many nodes can choose the same relay. LEST load-table Dijkstra coordinates them with two components: a sequential tree construction with within-round load booking, and a shared, tiered load table that supplies the rates.

*Sequential construction with within-round booking.* The sink processes nodes in increasing distance from the sink. For each node  $i$  that does not yet have a route, it runs Dijkstra from  $i$  to the sink over the cost

$$w(j, k) = [e_{\text{tx}}(j, k) + e_{\text{rx}}] \cdot \frac{E_0}{E_j} \cdot \left(1 + \kappa \frac{A_j}{\bar{r}}\right), \quad (7)$$

where  $A_j$  is the traffic already booked through node  $j$  in this round's construction,  $\bar{r}$  the mean node rate and  $\kappa$  the load weight. The whole path is then fixed. Every node on it that had no route takes its next hop from the path, and node  $i$ 's rate is added to  $A_j$  for every node  $j$  on the path. A node whose route is already fixed keeps its single next hop, so later paths that reach it follow its existing route and the result is always a tree. Later nodes therefore see busy relays as more expensive. The extra cost is proportional to the booked traffic, so a heavily used relay is avoided whenever a nearly equivalent alternative exists, while a relay with no good alternative is still used. With  $\kappa = 0$  the construction is exactly battery-weighted Dijkstra; a unit test checks this. Algorithm 1 gives the procedure and Fig. 1 an example.

*The LEST load table.* The rates come from a Lightweight State Table (LEST), a coordination pattern from our earlier work on fuzzy Q-learning, reused here without its learning layer and with load in place of energy. Node  $i$  measures its own rate  $r_i$  as its mean packets per round over the last 24 rounds and normalises it,  $x_i = 1 - r_i/(2r_{\text{max}})$ , where  $r_{\text{max}}$  is the largest per-node rate in the pre-deployment history. It maps  $x_i$  to one of four tiers  $q_i \in \{\text{Critical, Low, Medium, High}\}$  with boundaries 0.15, 0.40 and 0.75. The tier changes only when a boundary is crossed *and*  $x_i$  has moved by more than a hysteresis band of 0.05 since the previous report, so a node sitting on a boundary does not flap. The tier travels in one byte on data packets the node already sends, and the sink broadcasts all tiers, 2 bits per node, in the round's advertisement. Both costs are charged (Section 3.3). The sink decodes each tier to its interval midpoint,  $\tilde{r}_i = 2r_{\text{max}}(1 - \text{mid}(q_i))$ , and uses  $\tilde{r}_i$  for booking. Energy is *not* tiered: Eq. (7) uses the exact battery, as battery-weighted Dijkstra does. Section 6.3 shows why: with the original LEST trigger a slowly draining battery never moves by more than the band between two reports, so its tier never changes.

*Complexity.* Each Dijkstra run on the dense graph costs  $O(n^2)$ , and at most one run is made per node that lacks a route when it is reached, so a round

---

**Algorithm 1** LEST load-table Dijkstra: one round at the sink

---

**Require:** alive nodes  $V$ , batteries  $E_j$ , decoded LEST rates  $\tilde{r}_j$ , link energies  $e_{\text{tx}}(j, k)$ ,  $e_{\text{rx}}$ , load weight  $\kappa$

**Ensure:** parent function  $p$  (the round's routing tree)

```
1:  $A_j \leftarrow 0$  and  $p(j) \leftarrow \perp$  for all  $j \in V$ ;  $\bar{r} \leftarrow$  mean of  $\tilde{r}_j$  over  $V$ 
2: for  $i \in V$  in increasing distance to the sink do
3:   if  $p(i) \neq \perp$  then continue ▷ already on an earlier path
4:   end if
5:    $m_j \leftarrow (E_0/E_j)(1 + \kappa A_j/\bar{r})$  for all  $j$ 
6:   build the graph with arc weights  $[e_{\text{tx}}(j, k) + e_{\text{rx}}] m_j$ ; for every  $j$  with  $p(j) \neq \perp$  keep only the arc  $j \rightarrow p(j)$ 
7:    $P \leftarrow$  Dijkstra least-cost path from  $i$  to the sink
8:   for each node  $u$  on  $P$ , excluding the sink do
9:     if  $p(u) = \perp$  then  $p(u) \leftarrow$  successor of  $u$  on  $P$ 
10:    end if
11:     $A_u \leftarrow A_u + \tilde{r}_i$  ▷ book  $i$ 's traffic on its path
12:  end for
13: end for
14: return  $p$ 
```

---

costs  $O(n^3)$  in the worst case. In practice many nodes are fixed by earlier paths. The measured time is about 25 ms per round at  $n = 100$  on one CPU core, against under 1 ms for battery-weighted Dijkstra and about 0.2 s for one LP solve. Faster shortest-path algorithms [11] would change only this time, not the tree.

#### 4.3. Upper reference: the LP planner

To measure what full flow optimisation adds, the sink re-solves the LP (3)–(4) every round. The current batteries  $E_i$  replace  $E_0$ , and each node generates  $L\hat{r}_i$  bits per round, where  $\hat{r}_i$  is its expected rate over the next 24 rounds. A small total-energy term breaks ties among equally long-lived plans:

$$\max T - \mu \sum_{ij} f_{ij} (e_{\text{tx}}(i, j) + e_{\text{rx}})/E_0, \quad \mu = 10^{-3}. \quad (8)$$

The solution gives the fraction of each node's traffic to send to each next hop. In each round every node sends to one next hop, rotating among its LP next hops in proportion to those fractions (weighted round-robin), so each round is still a single tree. We call this configuration v2. Version v1 re-solves every 4 rounds without the tie-break. The rate  $\hat{r}_i$  comes from additive

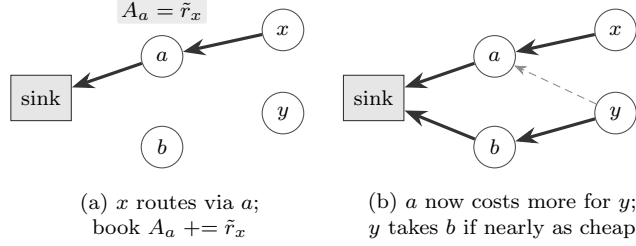


Figure 1: Within-round booking. Node  $x$  is routed first and its traffic is booked on relay  $a$ . When  $y$  is routed,  $a$ 's cost in Eq. (7) has risen by the factor  $1 + \kappa A_a / \bar{r}$ , so  $y$  takes relay  $b$  when its path is nearly as cheap. Battery-weighted Dijkstra would send both through  $a$ .

Holt-Winters [17, 18] with season length 24 ( $\alpha = 0.05$ ,  $\beta = 0$ ,  $\gamma = 0.1$ ; on Intel data  $\gamma = 0.3$ ), updated with the traffic the sink has just received. The ablation replaces it with a fixed historical mean. The LP is solved with HiGHS [20].

#### 4.4. Other reference methods

*MADII rebuild.* No code was released, so the network, the reward, the metrics and the two-stage training of [2] were rebuilt in PyTorch: an Informer encoder ( $d_{\text{model}} = 128$ , 2 layers, 4 heads), 1,000 episodes (300 imitation episodes from FCM, HBA and POA in rotation, then 700  $\epsilon$ -greedy), Adam with learning rate  $10^{-4}$ , and Double DQN [21] over 101 candidate next hops. The paper does not say how per-sensor Q-values form one training target. Averaging them before the loss let the network collapse to direct transmission, so each sensor's chosen value is regressed on the shared round return instead. Because all 100 sensors explore at once, the paper's  $\epsilon$  schedule ( $1.0 \rightarrow 0.1$ ) randomises the whole network, so its shape is kept and scaled to  $0.1 \rightarrow 0.01$ . To check that these two choices do not explain the result, we also ran the paper's schedule and the paper's full model size (27.3 M parameters). Both landed in the same 8–11% band (Section 6.1). MADII was trained on constant traffic and is not retrained for the time-varying scenarios.

*EBR-DA, as specified.* The node weight is  $0.6(1 - E/E_0)^2 + 0.4b^2$ , with  $b$  the buffer occupancy, over a hop tree with an 80 m radius [7], and each node forwards to its lightest neighbour one hop nearer the sink. Our model has no buffers, so occupancy is a node's traffic relative to the busiest node's. A node with no neighbour in range forwards to its nearest node nearer the sink (bridging); without bridging, isolated nodes die within a few rounds.

Table 4: Characteristics of the evaluated algorithms ( $n = 100$ ; times on one CPU core).

| Algorithm                                           | Decided by                   | Information used             | Work per round                      | Training       | Extra control bits          | Time per round  |
|-----------------------------------------------------|------------------------------|------------------------------|-------------------------------------|----------------|-----------------------------|-----------------|
| Min-energy Dijkstra                                 | sink                         | positions                    | one Dijkstra                        | none           | 0                           | < 1 ms          |
| Battery-weighted Dijkstra                           | sink                         | + exact batteries            | one Dijkstra                        | none           | 0                           | < 1 ms          |
| <b>LEST</b><br><b>load-table</b><br><b>Dijkstra</b> | sink                         | + 4-tier load per node       | $\leq n$ Dijkstra runs              | none           | 8/data packet + $2n$ /round | $\approx 25$ ms |
| LP planner v2 (reference)                           | sink                         | batteries + forecast rates   | one LP (HiGHS)                      | HW fit         | 0                           | $\approx 0.2$ s |
| Predictive / price-guided Dijkstra                  | sink                         | + forecast or LP prices      | one Dijkstra (+ daily LP)           | HW fit         | 0                           | < 1 ms          |
| EBR-DA [7]                                          | sink, per node rule          | energy, buffer of neighbours | hop tree + local choice             | none           | 0 (not charged)             | < 1 ms          |
| MADII [2]                                           | sink (centralised execution) | node states                  | Informer over $100 \times 101$ hops | 1,000 episodes | 0                           | GPU in [2]      |

*Dijkstra variants (negative results).* We also test the following variants:

- *Predictive Dijkstra.* Each battery is reduced by a reserve for the extra load that Holt-Winters expects,  $E_{\text{eff},i} = \max(E_i - \lambda R_i, 0.05E_i)$ .
- *Lifetime Dijkstra.* The cost is weighted by forecast load or by forecast rounds left.
- *Receiver-weighted Dijkstra.* The receive cost is weighted by the receiver’s battery.
- *Price-guided Dijkstra.* The cost is scaled by the LP’s dual energy prices, refreshed daily.
- *RL warm-started from Dijkstra.* A 145-parameter network multiplies Eq. (6) by  $\exp(m_\theta(\phi_i))$  and is trained by PGPE [22]. Its zero initial output makes the untrained policy exactly battery Dijkstra.

Each variant’s settings were tuned on validation deployments (Section 5).

#### 4.5. Characteristics of the evaluated algorithms

Table 4 summarises what each evaluated router needs.

## 5. Evaluation protocol

*Deployments and data splits.* The held-out test set has 30 deployments (seeds 0–29) per scenario. All tuning uses separate validation deployments (seeds 500–511; for the RL warm start, 500–519), and RL training uses seeds of 1,000 and above. On the Intel Lab bench, batteries, Holt-Winters and the send-on-delta thresholds were set on the first 14 days. The test then ran once on 27 deployments starting every 8 hours in the later days. Every method sees the same deployments and traffic (A14).

*Tuning.* Each method’s free settings were chosen on validation deployments, and only the best setting was run on the test set. For LEST Dijkstra,  $\kappa \in \{0.03, \dots, 10\}$  and two build orders were tried; nearest-first with  $\kappa = 1$  was best. The LEST tiers, band and trigger were taken unchanged from the original LEST design, not tuned. For the LP planner, re-solving every 8, 4, 2 and 1 rounds gave 86.9%, 91.5%, 94.1% and 96.8% of the oracle on validation, and the tie-break added up to 0.7 points. Two candidate changes were rejected on validation:

- Carrying round-robin credits across re-solves scored 81.1%, because each re-solve already corrects past deviations.
- A burst-triggered re-solve fired on Poisson noise in 70–75% of rounds.

The Dijkstra variants and EBR-DA’s reconstructed cost were tuned the same way (32 settings for EBR-DA).

*Metrics.* The primary metric is FND as a percentage of the deployment’s ceiling. We report its mean, its worst case over deployments, and wins/ties/losses (W/T/L) against battery-weighted Dijkstra, deployment by deployment. For the time-varying scenarios, the mean  $S$  weights R, N and B equally. D is reported beside it.

*Statistical testing.* All comparisons are paired by deployment, so we use the two-sided Wilcoxon signed-rank test [23], as recommended for comparing algorithms over multiple problem instances [24]. The test makes no normality assumption, and lifetime ratios are bounded and skewed. Zero differences, where a method produces exactly the same lifetime, are counted as ties and dropped, following Wilcoxon. Within each family of comparisons the  $p$ -values are adjusted with the Holm procedure [25]. As an effect size we report

Table 5: Reproduction check against MADII’s printed results (30 deployments; FND in rounds).

| Method           | Rebuild | Printed [2] | Ratio |
|------------------|---------|-------------|-------|
| MADII (Informer) | 19.5    | 19          | 1.03  |
| HBA clustering   | 12.4    | 14          | 0.89  |
| POA clustering   | 10.9    | 13          | 0.84  |
| FCM clustering   | 5.4     | 19          | 0.28  |

the matched-pairs rank-biserial correlation  $r$  [26], which equals  $+1$  when a method is better on every deployment. We use a significance level of 0.05 after adjustment.

## 6. Results

### 6.1. Static traffic in MADII’s setting

*Reproduction check.* Table 5 compares our rebuild with MADII’s printed results. The headline model matches, with 19.5 rounds to FND against the paper’s 19. Two of the paper’s baselines that we could implement exactly, HBA and POA, land within about 15%. One mismatch remains: our FCM matches only at later death fractions. We also ran MADII exactly as published. With the paper’s  $\varepsilon$  schedule ( $1.0 \rightarrow 0.1$ ), training episodes collapsed to 1–1.6 rounds once exploration began, and the held-out FND was 14.9 rounds (8.2% of the ceiling). The paper’s full model size (27.3 M parameters, evaluated at episode 200 of 1,000) reached 15.4 rounds (8.4%). A model 90 times larger thus lands in the same band, and the rebuild matches the paper’s own figure, so MADII’s shortfall is not an artefact of our implementation choices.

*Every method against the optimum.* Table 6 and Fig. 2 give the lifetime of each method as a percentage of the LP ceiling (mean  $T^* = 169.7$  rounds). Battery-weighted Dijkstra keeps every node alive for 151 rounds on average, 88.4% of the optimum, against 19.5 rounds (11.1%) for MADII: about  $8\times$  longer, with no training. It differs from min-energy Dijkstra (24.3%) by a single factor, the battery weighting. Every one of the 30 deployments under battery Dijkstra lies between 80.8% and 92.6% of the optimum, whereas MADII stays between 3.9% and 19.3%. The gap therefore holds deployment by deployment, not only on average. LP-flow re-solved every 4 rounds reaches 94.5%. RL warm-started from battery Dijkstra gains 0.5 points, which is not significant ( $p = 0.24$ ).

Table 6: Static traffic in MADII’s setting: lifetime against the provable optimum (30 held-out deployments). W/T/L and  $p$  against battery-weighted Dijkstra (Wilcoxon signed-rank).

| Method                               | FND   | % of $T^*$ | Worst % | W/T/L, $p$                       |
|--------------------------------------|-------|------------|---------|----------------------------------|
| LP-flow adaptive (LP every 4 rounds) | 161.5 | 94.5       | 81.0    | 29/0/1, $p = 2.0 \times 10^{-7}$ |
| RL warm-started from Dijkstra        | 152.0 | 88.9       | 83.0    | 15/5/10, $p = 0.241$             |
| Battery-weighted Dijkstra            | 151.4 | 88.4       | 80.8    | reference                        |
| LP-flow static (solved once)         | 149.6 | 87.5       | 53.2    | 16/0/14, $p = 0.824$             |
| Min-energy Dijkstra                  | 42.5  | 24.3       | 14.2    | 0/0/30, $p = 1.9 \times 10^{-9}$ |
| MADII rebuild [2]                    | 19.5  | 11.1       | 3.9     | 0/0/30, $p = 1.9 \times 10^{-9}$ |
| Direct to sink                       | 8.3   | 4.4        | 2.9     | –                                |
| FCM clustering (MADII’s baseline)    | 5.4   | 2.6        | 0.0     | 0/0/30, $p = 1.9 \times 10^{-9}$ |

### 6.2. Time-varying traffic

Table 7 and Fig. 3 report the time-varying scenarios. LEST load-table Dijkstra reaches 90.5% of the oracle, against 85.0% for battery-weighted Dijkstra. It wins 118 of the 120 deployments across R, N, B and D, and it gains in every scenario, including the pattern shift that nothing was tuned on (90.0% against 83.0%). It closes about half of the gap between battery Dijkstra and the LP planner with no solver and no training. Sharing load through the 4-tier LEST table, with its overhead charged, costs 0.3 points against exact loads (90.5% against 90.8%). The LP planner v2 reaches 97.0% and beats battery Dijkstra on all 120 deployments, with a worst case of 92.1% against 74.3%. MADII, trained on constant traffic, stays at 13–16%, and EBR-DA as specified at 14%. The best of five ways of adding prediction or tuning to Dijkstra, LP-price guidance, gains 1.0 point, and the others stay at or below battery Dijkstra.

### 6.3. Ablation of LEST load-table Dijkstra

Table 8 removes each part of the method in turn.

- **Load only.** Without the battery weight, the load table alone reaches 45.1%. It spreads traffic evenly but keeps relaying through nodes that are running out.
- **Energy only.** Without the load table the method is battery-weighted Dijkstra, at 85.0%. It protects weak nodes but lets many nodes pile onto the same relay.

Table 7: Time-varying traffic: lifetime as a percentage of the oracle (30 held-out deployments per scenario; R: daily surge, N: constant, B: bursts, D: pattern shift). Mean  $S$  weights R, N and B equally; Worst is over those 90 deployments; D is not tuned on. W/T/L against battery-weighted Dijkstra over all 120 deployments.

| Method                                         | R    | N    | B    | Mean $S$ | Worst | D    | W/T/L    |
|------------------------------------------------|------|------|------|----------|-------|------|----------|
| <i>Proposed</i>                                |      |      |      |          |       |      |          |
| <b>LEST load-table Dijkstra</b>                | 89.5 | 92.6 | 89.3 | 90.5     | 76.8  | 90.0 | 118/1/1  |
| Load-aware Dijkstra, exact load (no LEST)      | 90.2 | 93.0 | 89.2 | 90.8     | 76.8  | 90.9 | 116/3/1  |
| <i>Upper reference (LP solver every round)</i> |      |      |      |          |       |      |          |
| LP planner v2, Holt-Winters                    | 96.4 | 98.5 | 96.2 | 97.0     | 92.1  | 95.6 | 120/0/0  |
| LP planner v2, historical mean (no forecast)   | 95.7 | 98.5 | 95.2 | 96.5     | 84.8  | 95.1 | 120/0/0  |
| LP planner v2, perfect forecast (diagnostic)   | 96.1 | 98.5 | 96.2 | 96.9     | 91.4  | 96.1 | 120/0/0  |
| LP planner v1 (every 4 rounds), Holt-Winters   | 89.8 | 97.4 | 88.3 | 91.8     | 74.6  | 87.9 | 105/4/11 |
| <i>Classical and published</i>                 |      |      |      |          |       |      |          |
| Battery-weighted Dijkstra                      | 83.2 | 88.6 | 83.1 | 85.0     | 74.3  | 83.0 | –        |
| Price-guided Dijkstra (daily LP prices)        | 84.6 | 89.2 | 84.3 | 86.0     | 76.3  | 84.0 | 66/22/32 |
| Receiver-weighted Dijkstra                     | 83.7 | 88.3 | 83.0 | 85.0     | 76.1  | 84.5 | 49/28/43 |
| Predictive Dijkstra (Holt-Winters reserve)     | 81.4 | 88.6 | 82.3 | 84.1     | 63.4  | 82.6 | 24/47/49 |
| Lifetime Dijkstra (forecast load)              | 82.3 | 88.6 | 81.2 | 84.0     | 68.6  | 82.4 | 27/48/45 |
| Static LP (solved once)                        | 60.7 | 87.8 | 57.8 | 68.7     | 37.3  | 59.7 | 16/1/103 |
| Reactive LP (last 4 rounds' traffic)           | 52.5 | 97.4 | 50.4 | 66.8     | 22.2  | 52.5 | 32/1/87  |
| EBR-DA [7], as specified                       | 14.0 | 13.5 | 14.1 | 13.9     | 0.5   | 14.0 | 0/0/120  |
| EBR-DA [7], isolated nodes bridged             | 15.2 | 14.9 | 15.3 | 15.1     | 0.5   | 15.2 | 0/0/120  |
| <i>Learned</i>                                 |      |      |      |          |       |      |          |
| MADII [2] (trained on constant traffic)        | 13.0 | 15.5 | 15.6 | 14.7     | 2.9   | 13.6 | 0/0/120  |

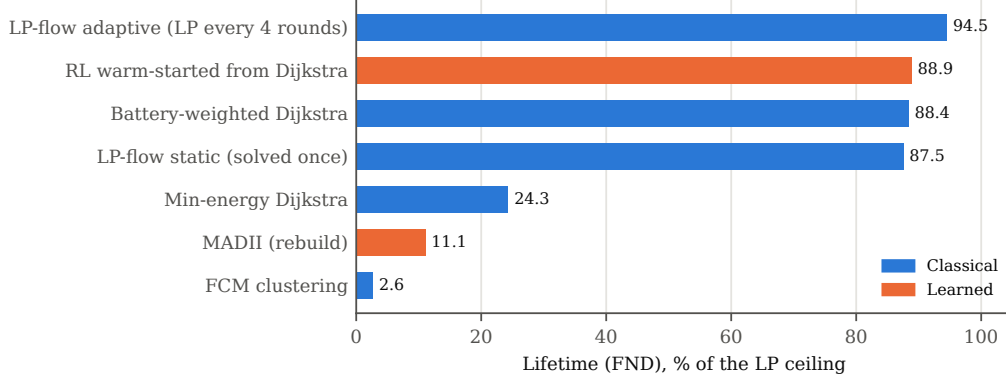


Figure 2: Static traffic in MADII’s setting: mean lifetime to first node death as a percentage of the LP ceiling (30 held-out deployments).

- **Energy in tiers.** Compressing energy into LEST tiers as well lowers validation lifetime to 43.8% with the original trigger. A slowly draining battery never moves by more than the 0.05 band between two reports, so its tier never changes (zero tier changes in a test run). With a standard Schmitt trigger it reaches 79.2%, and it needs 32 levels or more to recover 89%.
- **Load in tiers.** The original trigger works for load: rates jump from round to round, so a tier changes in only 0.19% of node-rounds. Finer load tables (8 or 16 levels) gain at most 0.2 points.

EBR-DA also mixes energy and load [7], but with a squared term that is linear in energy and over a minimum-hop tree. It reaches 15% as specified. With its cost retuned on our bench it reaches 50.3%, still far below the sharp  $E_0/E_i$  weight combined with load booked within the round.

#### 6.4. What prediction adds

*To the LP planner.* With the same v2 planner, a fixed historical rate reaches 96.5% against 97.0% with Holt-Winters. The difference is +0.7 points under surges (17/6/7,  $p = 0.047$  unadjusted,  $p = 0.19$  after Holm correction), zero with constant traffic, and not significant under bursts or the pattern shift. The forecast’s clearest benefit is the worst case (92.1% against 84.8%). Holt-Winters already matches a perfect forecast (96.9%), so a better forecaster could not add more. The gain comes from the planner: moving from v1

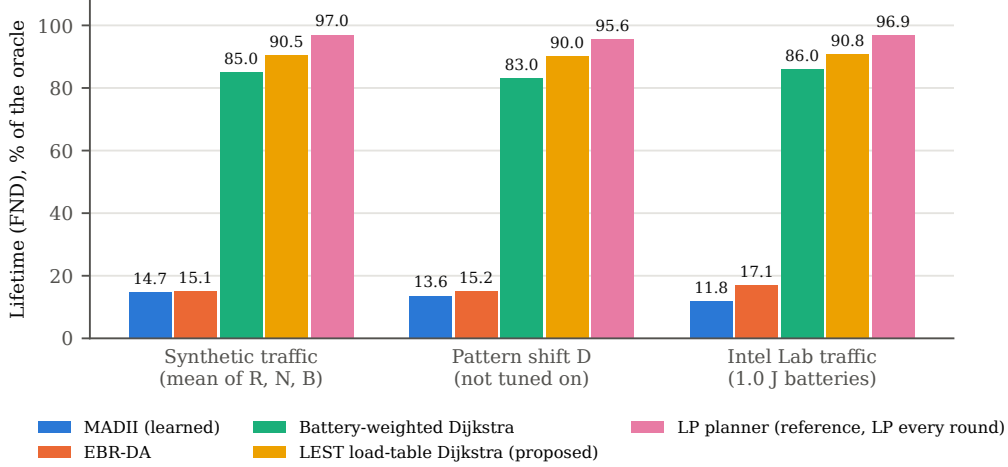


Figure 3: Lifetime as a percentage of the oracle on synthetic traffic, under a pattern shift and on real Intel Lab traffic.

(every 4 rounds) to v2 (every round) adds 6.6, 1.1, 7.8 and 7.7 points in R, N, B and D. Re-solving from measured batteries corrects a wrong rate through the batteries themselves. What matters is a rate averaged over a full day. A reactive LP that plans with the last 4 rounds’ traffic collapses to 50–53% under surges, and persistence to 13%.

*To Dijkstra.* Prediction, lifetime weighting, exponent tuning and LP prices all stay within about  $\pm 1$  point of battery Dijkstra (Table 7). Stronger battery exponents are worse:  $\alpha = 2, 5, 20$  and  $50$  give 77.4% down to 10.7% on validation. LP prices fed back every round collapsed to 6–18% on validation. A tracing study explains predictive Dijkstra’s loss under surges. One hour before each surge, the 20% of nodes with the highest forecast are exactly the hotspot nodes in every deployment, so the forecast senses the pattern correctly. Predictive Dijkstra does move relay traffic off those nodes before the surge (from 30.2% to 28.2% of relayed packets). Once the surge starts, however, the reserve vanishes, the spared nodes look rich, their relay share rises (43.6% to 46.8%), and a hotspot node dies first in 67% of deployments against 47%. Battery Dijkstra already charges every relay its energy per packet divided by its battery. Any further per-node signal moves whole subtrees onto neighbours, which then die first. LEST Dijkstra avoids this because its booking is incremental within the round.

Table 8: Ablation of LEST load-table Dijkstra (% of the oracle; held-out: mean of R, N, B over 90 deployments; validation: seeds 500–511). Control bits per alive node per round in the last column.

| Variant                                    | Held-out    | Validation  | Ctrl bits     |
|--------------------------------------------|-------------|-------------|---------------|
| <b>Exact energy + LEST load (proposed)</b> | <b>90.5</b> | 90.7        | 300           |
| Exact energy + exact load                  | 90.8        | 91.1        | 100           |
| Exact energy only (battery Dijkstra)       | 85.0        | 84.6        | 100           |
| Load only (no battery weight)              | 45.1        | 46.7        | 100           |
| Exact energy + LEST load, Schmitt trigger  | –           | 90.4        | 300           |
| Exact energy + 8 / 16-level load           | –           | 90.9 / 90.7 | 400 / 500     |
| Energy also in 4 tiers, original trigger   | –           | 43.8        | 400           |
| Energy also in 4 tiers, Schmitt trigger    | –           | 79.2        | 400           |
| Energy also in 32 / 256 levels             | –           | 89.0 / 89.1 | 1,200 / 1,600 |
| EBR-DA as specified [7]                    | 13.9        | –           | 100           |
| EBR-DA cost, retuned on this bench         | 50.3        | 53.3        | 100           |

### 6.5. Real traffic: Intel Berkeley Lab

Table 9 shows that the ranking found on synthetic traffic holds on real traffic. At  $E_0 = 1.0$  J, LEST Dijkstra reaches 90.8% against 86.0% for battery Dijkstra (16/7/4), and the LP planner reaches 96.9%. EBR-DA reaches 17.1% and MADII 11.8%. The forecast is far more accurate than the historical mean here (next-day rate error 0.52 against 1.03 packets per mote-hour), yet the lifetime difference is only 0.3 points, and its visible benefit is again the worst case (88.8% against 79.9%). At  $E_0 = 1.5$  J, about half the deployments outlive the dataset and count as ties, so the 1.0 J run is the cleaner comparison.

### 6.6. Statistical tests

Table 10 reports the paired Wilcoxon signed-rank tests for the main comparisons, and Fig. 4 shows the deployment-by-deployment pairs behind the central one.

- **LEST Dijkstra against battery Dijkstra.** LEST Dijkstra is better on time-varying traffic (median +4.6 points,  $p_{\text{Holm}} = 2.2 \times 10^{-20}$ ,  $r = +1.00$ ) and on Intel Lab traffic (median +0.5,  $p_{\text{Holm}} = 0.005$ ,  $r = +0.81$ ).
- **LP planner against LEST Dijkstra.** The LP planner is better again in both settings.

Table 9: Intel Lab traffic: lifetime as a percentage of the oracle (27 test deployments). W/T/L against battery-weighted Dijkstra.

| Method                          | $E_0 = 1.0$ J |       |        | $E_0 = 1.5$ J |       |         |
|---------------------------------|---------------|-------|--------|---------------|-------|---------|
|                                 | Mean          | Worst | W/T/L  | Mean          | Worst | W/T/L   |
| <b>LEST load-table Dijkstra</b> | 90.8          | 77.3  | 16/7/4 | 94.2          | 69.3  | 10/16/1 |
| Load-aware Dijkstra, exact load | 90.3          | 68.1  | 19/7/1 | 94.7          | 76.3  | 11/14/2 |
| LP planner v2, Holt-Winters     | 96.9          | 88.8  | 22/5/0 | 98.0          | 86.7  | 14/13/0 |
| LP planner v2, historical mean  | 96.6          | 79.9  | 22/5/0 | 98.3          | 86.7  | 14/13/0 |
| LP planner v2, perfect forecast | 96.3          | 79.0  | 22/5/0 | 98.3          | 87.7  | 14/13/0 |
| LP planner v1, Holt-Winters     | 93.7          | 68.1  | 20/7/0 | 96.7          | 79.5  | 13/14/0 |
| Battery-weighted Dijkstra       | 86.0          | 63.0  | –      | 93.3          | 69.7  | –       |
| Reactive LP                     | 85.4          | 51.1  | 13/7/7 | 94.2          | 48.3  | 11/14/2 |
| Predictive Dijkstra             | 84.1          | 51.1  | 4/9/14 | 89.3          | 55.3  | 4/14/9  |
| Static LP (solved once)         | 54.0          | 16.7  | 0/0/27 | 62.8          | 16.7  | 0/5/22  |
| EBR-DA, isolated nodes          | 17.1          | 10.6  | 0/0/27 | 18.8          | 11.8  | 0/0/27  |
| bridged                         |               |       |        |               |       |         |
| MADII (not retrained)           | 11.8          | 0.5   | 0/0/27 | 17.2          | 7.1   | 0/0/27  |

- **LEST against exact loads.** The 4-tier table loses a small but significant 0.5 points against exact loads on synthetic traffic ( $p = 6.4 \times 10^{-4}$ ), the price of sharing load in 2 bits.
- **Prediction and learning on Dijkstra.** Predictive Dijkstra is significantly *worse* than battery Dijkstra on synthetic traffic ( $p = 0.007$ ). RL warm-started from Dijkstra is not significantly different from it ( $p = 0.24$ ).

Every adjusted  $p$  for MADII and EBR-DA is below  $10^{-6}$ , with  $r = -1$ .

### 6.7. Traffic metrics and computation

Lifetime is not bought at the expense of delivery (Table 11). In MADII’s static setting, the delivery ratio is about 99% for every router. LEST Dijkstra delivers more data by the time half the nodes are dead (HND) than battery Dijkstra, and more per joule. MADII delivers 2.7 times less data by HND and about half as much per joule. Its one advantage is shorter paths (2.61 hops), obtained from long direct links that drain batteries. Our rebuild delivers 25,349 kbit by HND against the paper’s printed 25,708, a further check on the reproduction. In computation, battery Dijkstra needs under 1 ms per round, LEST Dijkstra about 25 ms and the LP planner about 0.2 s, all at the mains-powered sink, while MADII needs 1,000 training episodes before use (Table 4).

Table 10: Paired Wilcoxon signed-rank tests (two-sided). Mean and median are the paired differences, first method minus second, in points of the oracle percentage. Holm adjustment is applied within each setting’s family of comparisons;  $r$  is the matched-pairs rank-biserial correlation.

| Comparison                                                                       | Mean  | Median | W/T/L    | $p$                   | $p_{\text{Holm}}$     | $r$   |
|----------------------------------------------------------------------------------|-------|--------|----------|-----------------------|-----------------------|-------|
| <i>Time-varying traffic, <math>R+N+B+D</math> (120 deployments)</i>              |       |        |          |                       |                       |       |
| LEST Dijkstra vs battery Dijkstra                                                | +5.9  | +4.6   | 118/1/1  | $3.5 \times 10^{-21}$ | $2.2 \times 10^{-20}$ | +1.00 |
| LEST Dijkstra vs load-aware Dijkstra, exact load                                 | -0.5  | -0.5   | 30/29/61 | $6.4 \times 10^{-4}$  | $6.4 \times 10^{-4}$  | -0.41 |
| LP planner v2 vs LEST Dijkstra                                                   | +6.3  | +5.6   | 119/0/1  | $2.1 \times 10^{-21}$ | $2.2 \times 10^{-20}$ | +1.00 |
| LP planner v2 vs battery Dijkstra                                                | +12.2 | +12.0  | 120/0/0  | $2.0 \times 10^{-21}$ | $2.2 \times 10^{-20}$ | +1.00 |
| Price-guided vs battery Dijkstra                                                 | +1.0  | +0.6   | 66/22/32 | $2.2 \times 10^{-4}$  | $4.5 \times 10^{-4}$  | +0.43 |
| Predictive vs battery Dijkstra                                                   | -0.8  | +0.0   | 24/47/49 | 0.007                 | 0.007                 | -0.36 |
| Load table only vs LEST Dijkstra                                                 | -45.0 | -45.6  | 0/0/120  | $2.0 \times 10^{-21}$ | $2.2 \times 10^{-20}$ | -1.00 |
| EBR-DA vs LEST Dijkstra                                                          | -75.2 | -75.6  | 0/0/120  | $2.0 \times 10^{-21}$ | $2.2 \times 10^{-20}$ | -1.00 |
| MADII vs LEST Dijkstra                                                           | -76.0 | -77.1  | 0/0/120  | $2.0 \times 10^{-21}$ | $2.2 \times 10^{-20}$ | -1.00 |
| <i>Intel Lab traffic, <math>E_0 = 1.0</math> <math>J</math> (27 deployments)</i> |       |        |          |                       |                       |       |
| LEST Dijkstra vs battery Dijkstra                                                | +4.7  | +0.5   | 16/7/4   | 0.002                 | 0.005                 | +0.81 |
| LP planner v2 vs LEST Dijkstra                                                   | +6.2  | +6.5   | 21/6/0   | $9.5 \times 10^{-7}$  | $7.6 \times 10^{-6}$  | +1.00 |
| LP planner v2 vs battery Dijkstra                                                | +10.9 | +9.1   | 22/5/0   | $4.8 \times 10^{-7}$  | $4.3 \times 10^{-6}$  | +1.00 |
| Predictive vs battery Dijkstra                                                   | -1.9  | -0.4   | 4/9/14   | 0.090                 | 0.180                 | -0.46 |
| EBR-DA vs LEST Dijkstra                                                          | -73.7 | -72.8  | 0/0/27   | $1.5 \times 10^{-8}$  | $1.6 \times 10^{-7}$  | -1.00 |
| MADII vs LEST Dijkstra                                                           | -78.9 | -76.6  | 0/0/27   | $1.5 \times 10^{-8}$  | $1.6 \times 10^{-7}$  | -1.00 |
| <i>Static traffic, MADII’s setting (30 deployments)</i>                          |       |        |          |                       |                       |       |
| RL warm-started vs battery Dijkstra                                              | +0.5  | +0.3   | 15/5/10  | 0.241                 | 0.482                 | +0.27 |
| LP-flow adaptive vs battery Dijkstra                                             | +6.1  | +6.4   | 29/0/1   | $2.0 \times 10^{-7}$  | $6.1 \times 10^{-7}$  | +0.94 |
| MADII vs battery Dijkstra                                                        | -77.3 | -77.2  | 0/0/30   | $1.9 \times 10^{-9}$  | $1.1 \times 10^{-8}$  | -1.00 |

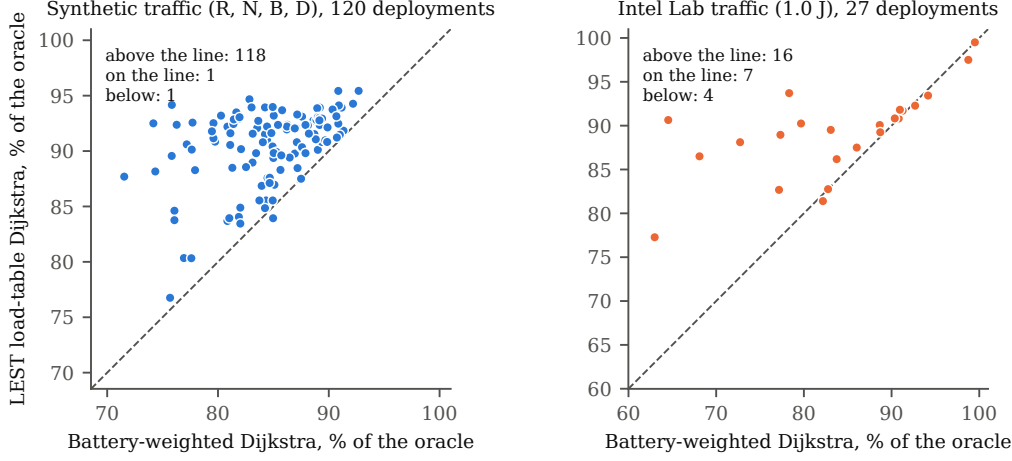


Figure 4: Deployment-by-deployment lifetime of LEST load-table Dijkstra against battery-weighted Dijkstra. Points above the dashed line are deployments where LEST Dijkstra lives longer.

## 7. Discussion

*Why MADII falls short.* There are three reasons, and none of them requires a bug in our rebuild.

- **The imitation stage does not teach the expert’s behaviour.** It only fills the replay memory with expert decisions for Q-learning, and the network is never trained to reproduce them. Consistent with this, a run that used battery Dijkstra (a 151-round expert) as the imitation source fell from 7.6 to 4.4 rounds between episodes 25 and 50; this run was stopped early.
- **The Informer has nothing to forecast.** It was designed for long time-series forecasting [15]. MADII feeds it a single snapshot of 101 nodes and asks it to route, and its sparse attention gains nothing at that size.
- **No baseline revealed the gap.** MADII is compared only with clustering, metaheuristic and learned methods, so the distance to shortest-path routing and to the optimum went unnoticed.

The broader lesson concerns evaluation practice. A learned router should be reported against a battery-weighted shortest-path router and against a

Table 11: Traffic metrics in MADII’s setting (constant traffic, 30 deployments). HND: rounds until half the nodes are dead. This run uses the time-varying simulator with constant traffic. The classical routers give the same FND as in Table 6, but the MADII rebuild gives 27.3 rounds rather than 19.5. Either value is below 17% of the ceiling.

| Router               | FND   | HND   | kbit by HND | kbit/J | Deliv. | Hops |
|----------------------|-------|-------|-------------|--------|--------|------|
| LP planner v2        | 167.7 | 170.7 | 67,475      | 1,355  | 99.1%  | 3.23 |
| <b>LEST Dijkstra</b> | 157.8 | 163.7 | 64,686      | 1,306  | 99.1%  | 3.34 |
| Battery Dijkstra     | 151.4 | 162.9 | 63,843      | 1,286  | 98.9%  | 3.53 |
| Min-energy Dijkstra  | 42.5  | 159.1 | 55,735      | 1,336  | 99.3%  | 3.96 |
| MADII rebuild        | 27.3  | 73.3  | 25,349      | 681    | 99.2%  | 2.61 |
| MADII, printed [2]   | 19    | 89    | 25,708      | –      | –      | –    |

provable ceiling. Both are cheap to compute, and together they place any new method on an absolute scale.

*Why coordination, and why exact energy.* Battery-weighted Dijkstra already captures most of the lifetime available from a single tree per round. What it lacks is coordination: nodes choose independently and overload shared relays. Booking load within the round supplies that coordination without a solver. Each extra unit of booked traffic raises a relay’s cost smoothly, so close decisions tip towards alternatives while necessary relays are still used. The ablation shows that the two signals correct each other. Energy without load lets nodes pile onto one relay, and load without energy keeps relaying through nearly empty nodes. Energy must stay exact for a different reason: it drains slowly, so any reporting rule that is robust to noise, such as LEST’s hysteresis trigger, freezes it. Load jumps from round to round and survives compression into 2 bits. The same freeze explains an observation from our earlier fuzzy-Q project, where every node remained in the same energy tier for the whole run.

*When to use which router.* If a sink can afford one LP solve per round (about 0.2 s at  $n = 100$ ), the LP planner is the best choice, at 97% of the oracle. If it cannot, LEST Dijkstra reaches about 90% with shortest paths alone, and battery Dijkstra about 85% with a single Dijkstra run. A traffic forecast is a small robustness refinement for the planner and does not help Dijkstra. In this setting, learning brings no benefit we could measure.

### 7.1. Threats to validity

*Internal validity.* MADII is our rebuild from the paper’s text. It reproduces the headline lifetime and delivered data, but it is not the authors’ code, which has not been released; we have requested it. Two harnesses give MADII different FND values (19.5 and 27.3 rounds); both are below 17% of the ceiling, and the classical routers agree across harnesses. EBR-DA is evaluated outside the aggregation setting it was designed for, with buffer occupancy approximated by load. All tuned settings were chosen on validation deployments, and the test sets were run once.

*Construct validity.* FND is one of several lifetime definitions. We also report 10%, 25% and 50% dead, delivered data and energy efficiency (Table 11), and the ranking is the same.

*External validity.* The radio model is the idealised one used by MADII (A3–A6): no collisions, no lossy links, no barriers and no idle-listening energy. Absolute percentages may shift under a packet-level MAC with retransmissions. Lossy links would raise the cost of long hops and of busy relays alike, which we expect to favour load-aware routing, but this has not been tested. The synthetic traffic models are designed by us. The Intel layout is scaled, its traffic is derived from readings rather than measured, and test deployments overlap in time. Networks of 54 and 100 nodes are tested; the LP grows with the square of  $n$ , and LEST Dijkstra’s worst case with its cube. Both LEST Dijkstra and the LP planner compute routes at the sink. A fully distributed variant, in which every node computes the same tree from a shared snapshot, would also need every battery level at every node, and it has not been tested.

## 8. Conclusion

Measured against a provable optimum, energy-efficient WSN routing falls into clear levels. A recent multi-agent DRL router and a published energy-and-load router reach about 15% of the ceiling. Battery-weighted Dijkstra reaches about 85% with no training. LEST load-table Dijkstra reaches about 90% with no solver. An LP planner re-solved every round reaches 97%. LEST load-table Dijkstra builds each round’s tree node by node, books every source’s traffic on its path, and shares load through a 4-tier table whose overhead it pays for. It wins 118 of 120 time-varying deployments against

battery Dijkstra, and the ranking holds on real Intel Lab traffic. Prediction adds half a point to the planner and nothing to Dijkstra, and learning adds no measurable gain. For lifetime routing in this setting, coordination and planning are what matter.

Future work will cover four directions:

- a distributed LEST Dijkstra, in which every node computes the same tree from a shared snapshot of energy and load;
- a packet-level evaluation with lossy links taken from the Intel connectivity data, collisions and retransmissions, together with networks of 200–500 nodes;
- further published learned baselines, including MADII retrained on time-varying traffic or the authors’ code if it is released;
- traffic with stronger drift or weekly cycles, where prediction may matter more.

### **CRedit authorship contribution statement**

[**Author 1**]: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft. [**Author 2**]: Software, Validation, Data curation, Writing – review & editing. [**Supervisor**]: Supervision, Writing – review & editing.

### **Declaration of competing interest**

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### **Data availability**

The simulator, every router, the result files and the scripts that regenerate every table and figure are available at [repository URL]. The Intel Berkeley Lab data are publicly available [8].

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used [name of tool / service] in order to [reason]. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

## Acknowledgements

[Funding and acknowledgements, or “This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.”]

## References

- [1] J.-H. Chang, L. Tassiulas, Maximum lifetime routing in wireless sensor networks, *IEEE/ACM Transactions on Networking* 12 (4) (2004) 609–619.
- [2] J. Yang, W. Li, C. Li, L. Zhang, L. Liu, An energy-efficient and transmission-efficient adaptive routing algorithm using deep reinforcement learning for wireless sensor networks, *IEEE Internet of Things Journal* 12 (23) (2025) 50414–50426. doi:10.1109/JIOT.2025.3609624.
- [3] M. U. Younus, M. K. Khan, M. R. Anjum, S. Afridi, Z. A. Arain, A. A. Jamali, Optimizing the lifetime of software defined wireless sensor network via reinforcement learning, *IEEE Access* 9 (2021). doi:10.1109/ACCESS.2020.3046693.
- [4] Tang, et al., Two-tier reinforcement-learning transmit-power control for low-power wireless mesh networks, *Journal of Network and Computer Applications* 239 (2025) 104177, aUTHORS: complete the full author list and title from the published article.
- [5] Energy efficient data transmission in WSN for network lifetime enhancement, in: *Proc. 7th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, IEEE, 2026, pp. 183–189, aUTHORS: add the author list.

- [6] When simple model just works: Is network traffic classification in crisis?, arXiv:2506.08655, aUTHORS: add the author list (2025).
- [7] O. A. Mahdi, Y. R. B. Al-Mayouf, A. B. Ghazi, M. A. Mohammed, A. W. A. Wahab, M. Y. I. B. Idris, An energy-aware and load-balancing routing scheme for wireless sensor networks, *Indonesian Journal of Electrical Engineering and Computer Science* 12 (3) (2018) 1312–1319. doi:10.11591/ijeecs.v12.i3.pp1312-1319.
- [8] Intel Berkeley Research Lab, Intel lab data, <http://db.csail.mit.edu/labdata/labdata.html> (2004).
- [9] W. R. Heinzelman, A. Chandrakasan, H. Balakrishnan, Energy-efficient communication protocol for wireless microsensor networks, in: *Proc. 33rd Hawaii International Conference on System Sciences*, 2000.
- [10] E. W. Dijkstra, A note on two problems in connexion with graphs, *Numerische Mathematik* 1 (1959) 269–271.
- [11] R. Duan, J. Mao, X. Mao, X. Shu, L. Yin, Breaking the sorting barrier for directed single-source shortest paths, in: *Proc. 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025.
- [12] H. Dai, R. Han, A node-centric load balancing algorithm for wireless sensor networks, in: *Proc. IEEE Global Telecommunications Conference (GLOBECOM)*, 2003.
- [13] F. Shan, W. Liang, J. Luo, X. Shen, Network lifetime maximization for time-sensitive data gathering in wireless sensor networks, *Computer Networks* 57 (2013) 1063–1077. doi:10.1016/j.comnet.2012.12.005.
- [14] V. Mnih, et al., Human-level control through deep reinforcement learning, *Nature* 518 (2015) 529–533.
- [15] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: *Proc. AAAI Conference on Artificial Intelligence*, 2021.
- [16] Learning for routing: A guided review of recent developments and future directions, arXiv:2507.00218, aUTHORS: add the author list (2025).

- [17] C. C. Holt, Forecasting seasonals and trends by exponentially weighted moving averages, *International Journal of Forecasting* 20 (1) (2004) 5–10, reprint of the 1957 ONR memorandum.
- [18] P. R. Winters, Forecasting sales by exponentially weighted moving averages, *Management Science* 6 (3) (1960) 324–342.
- [19] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, in: *Proc. 35th International Conference on Machine Learning (ICML)*, 2018.
- [20] Q. Huangfu, J. A. J. Hall, Parallelizing the dual revised simplex method, *Mathematical Programming Computation* 10 (1) (2018) 119–142.
- [21] H. van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, in: *Proc. AAAI Conference on Artificial Intelligence*, 2016.
- [22] F. Sehnke, C. Osendorfer, T. Rückstieß, A. Graves, J. Peters, J. Schmidhuber, Parameter-exploring policy gradients, *Neural Networks* 23 (4) (2010) 551–559.
- [23] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [24] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *Journal of Machine Learning Research* 7 (2006) 1–30.
- [25] S. Holm, A simple sequentially rejective multiple test procedure, *Scandinavian Journal of Statistics* 6 (2) (1979) 65–70.
- [26] D. S. Kerby, The simple difference formula: An approach to teaching nonparametric correlation, *Comprehensive Psychology* 3 (2014).